



Ahmed Irshad Hussain

sardarahmed.irshad2@gmail.com

© COPYRIGHT ALL RIGHTS RESERVED

CLOUD DATA ENGINEERING:
MICROSOFT FABRIC

*(Incremental Load, Merge &
Partitioning)*

By: Ahmed Irshad Hussain



Data Engineering: Microsoft Fabric End to End Project (Continue)

Microsoft Fabric

Previously we have made the end to end Fabric Data Engineering Pipeline.

But the question is “Are we done now?”

Answer is “No!”. There’s still a lot to do yet!

So now, we will look at the most important concepts in Data Engineering,

1. Incremental Load
2. Merge
3. Partitioning

Incremental load, merge, and partitioning are foundational in modern data engineering because they move away from slow, expensive, and risky "full table reloads" toward efficient, scalable, and near real-time data pipelines. As data volumes grow from gigabytes to terabytes and petabytes, re-processing everything every day is unsustainable. These three techniques work together to solve this.

Incremental Load:

Instead of copying an entire database every day (Full Load), an Incremental Load only captures the data that has changed or been added since the last successful run.

Merge (Upsert):

A **Merge** operation (often called an "Upsert") is a logic-based command that decides whether to **Update** an existing record or **Insert** a new one. It prevents duplicate entries while keeping current information fresh.

Partitioning:

Partitioning is the process of physically dividing a massive table into smaller, more manageable pieces (partitions) based on a specific column, like a date or a region.

Microsoft Fabric

Now let's do practical work,

Before that, as we have already a dataset “**Sales_Data**”, so instead of full load to “**Bronze Layer**”, as we did earlier, we will load first “**Historical Data**” then we will move towards Incremental Load.

Why historical data?

Because incremental load only works if we have already some historical data!

For Example, look at our dataset, csv file, we have already date column in our dataset. So, we will use that.

A	B	C	
date	product	category	price
1/1/2022	Smartpho	Electronic	
1/1/2022	Laptop	Electronic	
1/2/2022	T-Shirt	Clothing	
1/3/2022	Headphor	Electronic	

We will first load data till march which that will be our “Historical Data” then we will load more data which will be known as “Incremental load”

Microsoft Fabric

Phase 1: First Load (Historical Data).

Step 1 (Read Data + Fix Date)

```
1 from pyspark.sql.functions import col, to_date
2
3 df = spark.read.csv('Files/Bronze/sales_data.csv', header=True, inferSchema=True)
4
5 df = df.withColumn("date", to_date(col("date"), "M/d/yyyy"))
```

[35] ✓ <1 sec - Command executed in 1 sec 38 ms by Ahmed Fabric on 5/1/2026, 5:05:18 PM PySpark (Python) ▾

Step 2 (Take Historical Slice)

We simulate that only data till March has arrived.

> ⚙ Spark jobs (2 of 2 succeeded) 📄 Resources ...

```
1 from pyspark.sql.functions import col, to_date
2
3 df = spark.read.csv('Files/Bronze/sales_data.csv', header=True, inferSchema=True)
4
5 df = df.withColumn("date", to_date(col("date"), "M/d/yyyy"))
6
7 historical_df = df.filter(col("date") <= "2022-03-31")
```

✓ 2 sec - Command executed in 1 sec 620 ms by Ahmed Fabric on 5/1/2026, 5:09:54 PM

Step 3 (Create Bronze Table)

```
6
7 historical_df = df.filter(col("date") <= "2022-03-31")
8
9 historical_df.write.format("delta") \
10     .mode("overwrite") \
11     .saveAsTable("bronze_sales")
```

✓ 6 sec - Command executed in 6 sec 435 ms by Ahmed Fabric on 5/1/2026, 5:11:55 PM

Step 4 (Verify): Showing last 6 rows only

87	2022-03-26	Jeans	Clothing	50	20	1000
88	2022-03-27	Smartphone	Electronics	600	9	5400
89	2022-03-28	Watch	Accessories	150	10	1500
90	2022-03-29	Coat	Clothing	100	5	500
91	2022-03-30	Headphones	Electronics	100	10	1000
92	2022-03-31	Smartphone	Electronics	600	11	6600

Microsoft Fabric

Phase 2: Incremental Load + Merge.

Currently we have data from January till March.
New data = April till June.

Step 1: Creating Data Slice (Incremental Load)

We simulate that only data till March has arrived.

Step 2: Applying MERGE (Core Logic)

What does this do, if matched 'update', else 'Insert'.

Result: No duplicates + Data Updated.

Step 3: Verify MERGE: Showing last 6 rows only

```
1  from pyspark.sql.functions import col
2
3  new_data_df = df.filter(
4      (col("date") > "2022-03-31") &
5      (col("date") <= "2022-06-30")
6  )
```

✓ <1 sec - Command executed in 1 sec 31 ms by Ahmed Fabric on 5/1/2026, 5:21:52 PM

```
8
9  from delta.tables import DeltaTable
10
11  target = DeltaTable.forName(spark, "bronze_sales")
12
13  (target.alias("old")
14   .merge(
15       new_data_df.alias("new"),
16       "old.date = new.date AND old.product = new.product AND old.category = new.category"
17   )
18   .whenMatchedUpdateAll()
19   .whenNotMatchedInsertAll()
20   .execute())
```

✓ 9 sec - Command executed in 9 sec 339 ms by Ahmed Fabric on 5/1/2026, 5:24:12 PM

178	2022-06-25	Smartwatch	Accessories	200	8	1600
179	2022-06-26	Jeans	Clothing	50	15	750
180	2022-06-27	Smartphone	Electronics	600	9	5400
181	2022-06-28	Watch	Accessories	150	10	1500
182	2022-06-29	Coat	Clothing	100	5	500
183	2022-06-30	Headphones	Electronics	100	15	1500

BOOM! See, data updated till June perfectly

Loading Data till December!

```

1 from pyspark.sql.functions import col
2
3 new_data_df = df.filter(
4     (col("date") > "2022-06-30") &
5     (col("date") <= "2022-09-30")
6 )
7
8
9 from delta.tables import DeltaTable
10
11 target = DeltaTable.forName(spark, "bronze_sales")
12
13 (target.alias("old")
14     .merge(
15         new_data_df.alias("new"),
16         "old.date = new.date AND old.product = new.product AND old.category = new.category"
17     )
18     .whenMatchedUpdateAll()
19     .whenNotMatchedInsertAll()
20     .execute())

```

6 sec - Command executed in 6 sec 544 ms by Ahmed Fabric on 5/1/2026, 5:38:17 PM

Loading from July till September

Verify

270	2022-09-25	Jeans	Clothing	50	20	1000
271	2022-09-26	Smartphone	Electronics	600	9	5400
272	2022-09-27	Watch	Accessories	150	5	750
273	2022-09-28	Coat	Clothing	100	5	500
274	2022-09-29	Headphones	Electronics	100	10	1000
275	2022-09-30	Smartphone	Electronics	600	11	6600

```

1 from pyspark.sql.functions import col
2
3 new_data_df = df.filter(
4     (col("date") > "2022-09-30") &
5     (col("date") <= "2022-12-31")
6 )
7
8
9 from delta.tables import DeltaTable
10
11 target = DeltaTable.forName(spark, "bronze_sales")
12
13 (target.alias("old")
14     .merge(
15         new_data_df.alias("new"),
16         "old.date = new.date AND old.product = new.product AND old.category = new.category"
17     )
18     .whenMatchedUpdateAll()
19     .whenNotMatchedInsertAll()
20     .execute())

```

6 sec - Command executed in 6 sec 201 ms by Ahmed Fabric on 5/1/2026, 5:43:31 PM

Loading from October till December

Verify

364	2022-12-26	Smartphone	Electronics	600	9	5400
365	2022-12-27	Watch	Accessories	150	5	750
366	2022-12-28	Coat	Clothing	100	5	500
367	2022-12-29	Headphones	Electronics	100	10	1000
368	2022-12-30	Smartphone	Electronics	600	11	6600
369	2022-12-31	Hoodie	Clothing	40	30	1200

Microsoft Fabric

Phase 3: Clean + Partitioning (Silver Layer).

Now, Transforming Bronze → Clean, consistent, optimized data

Step 1: Load Bronze Data:

Step 2: Remove Nulls: Removes incomplete rows

Step 3: Removes Duplicates: Even though MERGE handled duplicates, this is safety layer

```
1 df_clean = spark.table("bronze_sales")
```

✓ 2 sec - Command executed in 2 sec 499 ms by Ahmed Fabric on 5/1/2026, 11:11:22 PM

```
1 df_clean = df_clean.dropna()
```

✓ <1 sec - Command executed in 584 ms by Ahmed Fabric on 5/1/2026, 11:12:45 PM

```
1 df_clean = df_clean.dropDuplicates()
```

✓ 3 sec - Command executed in 3 sec 337 ms by Ahmed Fabric on 5/1/2026, 11:15:05 PM

Microsoft Fabric

Step 4: Standardize Text:

It will make, bags → BAGS & Bags → BAGS

```
1 from pyspark.sql.functions import col, upper
2
3 df_clean = df_clean.withColumn("category", upper(col("category")))
```

✓ <1 sec - Command executed in 1 sec 198 ms by Ahmed Fabric on 5/1/2026, 11:17:08 PM

```
1 df_clean.select("category").distinct().show()
```

27] ✓ <1 sec - Command executed in 955 ms by Ahmed Fabric on 5/1/2026, 11:20:20 PM

> Spark jobs (2 of 2 succeeded) Resources

```
...
+-----+
| category|
+-----+
| SHOES|
| ACCESSORIES|
| BAGS|
| ELECTRONICS|
| CLOTHING|
| SHOESSES|
| BGAS|
| CLOHTING|
+-----+
```

Verifying!

As you can see, still some
garbage data, let's fix it in Step
5

```
1 from pyspark.sql.functions import when
2
3 df_clean = df_clean.withColumn(
4     "category",
5     when(col("category") == "BGAS", "BAGS")
6     .when(col("category") == "CLOHTING", "CLOTHING")
7     .when(col("category") == "SHOESSES", "SHOES")
8     .otherwise(col("category"))
9 )
```

✓ <1 sec - Command executed in 462 ms by Ahmed Fabric on 5/1/2026, 11:23:36 PM

```
1 df_clean.select("category").distinct().show()
```

✓ 1 sec - Command executed in 1 sec 307 ms by Ahmed Fabric on 5/1/2026, 11:23:36 PM

> Spark jobs (3 of 3 succeeded) Resources Log

```
+-----+
| category|
+-----+
| SHOES|
| ACCESSORIES|
| BAGS|
| ELECTRONICS|
| CLOTHING|
+-----+
```

Verifying!

Now, it's clean

Step 5: Fixing Inconsistent Values:

Microsoft Fabric

Step 6: Recalculating Revenue (Optional):

If wants to ensure correctness!.

```
1 df_clean = df_clean.withColumn("revenue", col("price") * col("quantity"))
```

✓ <1 sec - Command executed in 420 ms by Ahmed Fabric on 5/1/2026, 11:28:07 PM

 Log

Step 7: Save Silver Table (With Patitioning):

```
1 df_clean.write.format("delta") \
2   .mode("overwrite") \
3   .option("overwriteSchema", "true") \
4   .partitionBy("category") \
5   .saveAsTable("silver_sales")
```

✓ 10 sec - Command executed in 9 sec 972 ms by Ahmed Fabric on 5/1/2026, 11:31:41 PM

>  Spark jobs (12 of 12 succeeded)  Resources  Log

Why this specific line?

Because I have already silver table (without partition), so now I'm changing the structure (With partition). Spark won't allow me unless I force it So that's why "overwriteSchema = true" is necessary

What Partitioning did,

The data is now physically store like:

silver_sales/

- category=BAGS/
- category=CLOTHING/
- category=SHOES/
- category=ELECTRONICS/

Verifying

```
1 spark.sql("SHOW PARTITIONS silver_sales").show()
```

✓ 1 sec - Command executed in 1 sec 834 ms by Ahmed Fabric on 5/1/2026, 11:49:22 PM

>  Spark jobs (2 of 2 succeeded)  Resources  Log

```
+-----+
|      partition|
+-----+
|category=ACCESSORIES|
|   category=BAGS|
|   category=CLOTHING|
|category=ELECTRONICS|
|   category=SHOES|
+-----+
```

Microsoft Fabric

Phase 4: Gold Layer (Business + Data Product).

Now, Transforming Silver → usable insights (for dashboards / ML / business)

Step 1: Load Silver Data:

Step 2: Creating Aggregations: This is CEO – Ready dataset.

Step 3: Saving Gold Table:

```
1 df_gold = spark.table("silver_sales")
```

✓ 2 sec - Command executed in 2 sec 744 ms by Ahmed Fabric on 5/1/2026, 11:58:28 PM

>  Log

```
1 from pyspark.sql.functions import sum
2
3 gold_df = df_gold.groupBy("category").agg(
4     sum("revenue").alias("total_revenue"),
5     sum("quantity").alias("total_quantity")
6 )
```

✓ 6 sec - Command executed in 605 ms by Ahmed Fabric on 5/1/2026, 11:59:51 PM

Why I did,
Converted row level data (Silver Layer)
into Business metric (Gold)

```
1 gold_df.write.format("delta") \
2     .mode("overwrite") \
3     .option("overwriteSchema", "true") \
4     .saveAsTable("gold_sales")
```

✓ 9 sec - Command executed in 9 sec 677 ms by Ahmed Fabric on 5/2/2026, 12:03:29 AM

>  Spark jobs (14 of 14 succeeded)  Resources  Log

Verifying

```
1 spark.table("gold_sales").show()
```

✓ 3 sec - Command executed in 3 sec 645 ms by Ahmed Fabric on 5/2/2026, 12:04:29 AM

>  Spark jobs (5 of 5 succeeded)  Resources

category	total_revenue	total_quantity
ELECTRONICS	509480	1439
CLOTHING	94350	2251
ACCESSORIES	101400	902
BAGS	20400	408
SHOES	21600	270

What I have built till now,

CSV → Bronze (Raw + Incremental + MERGE)

→ Silver (Clean + Partitioned)

→ Gold (Aggregated / Business-ready)

Now I have:

- Incremental pipeline
- Upsert logic (MERGE)
- Partitioned optimization
- Clean transformation
- Business aggregation

Next Step: Dashboard (Power BI)

Microsoft Fabric

Phase 5: Power BI (Dashboard).

As I have already a semantic model that connected to gold sales from my previous project.

Since I rebuilt the silver and gold layer, I have to make sure that my semantic model refreshes the latest data. So, for this I have to refresh it!

Step 1: Refresh the Semantic Model

Step 2: Open the dashboard

	Name	Status	Type	Task
	Ahmed_PowerBI_1		Semantic ...	—
	Ahmed_Sales_Lakehouse		Lakehouse	—
	Ahmed_Sales_Lakehouse		SQL analyt...	—
	Sales Dashboard		Report	—

Open

File

Refresh



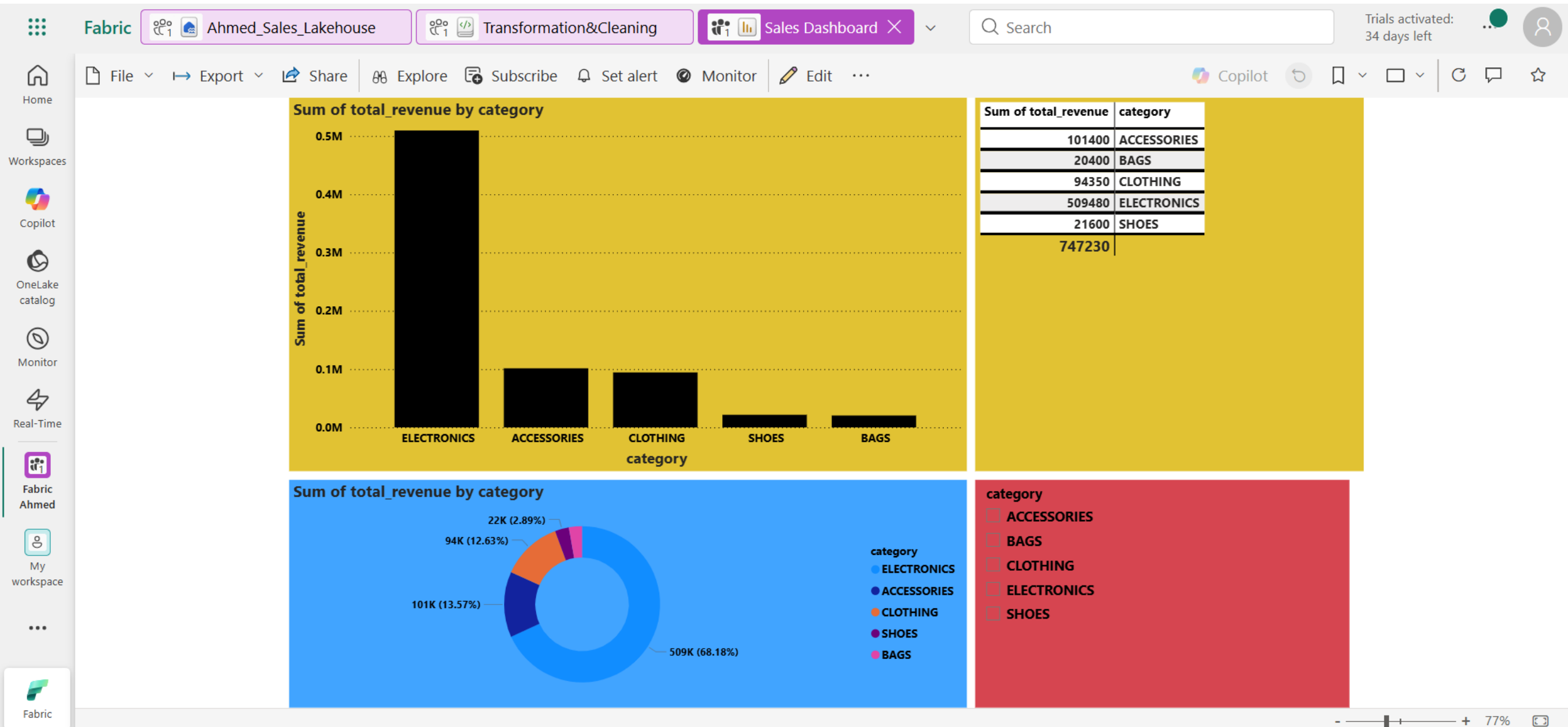
Semantic model
Ahmed_PowerBI_1

Choose the semantic model and then click refresh

	Ahmed_PowerBI_1	Semantic ...
	Ahmed_Sales_Lakehouse	Lakehouse
	Ahmed_Sales_Lakehouse	SQL analyt...
	Sales Dashboard	Report

After refreshing, click the dashboard

BOOM! The dashboard refreshed perfectly!



How to verify that the dashboard is truly connected with our pipeline?

Let's do the small changes and then refresh our dashboard!

```
1 df_test = spark.table("silver_sales")
2
3 df_test = df_test.withColumn("revenue", col("revenue") + 10)
4
5 df_test.write.format("delta") \
6     .mode("overwrite") \
7     .option("overwriteSchema", "true") \
8     .partitionBy("category") \
9     .saveAsTable("silver_sales")
```

✓ 7 sec - Command executed in 7 sec 973 ms by Ahmed Fabric on 5/2/2026, 5:21:19 PM

It will the data **Silver table (row-level data)**

The second line, It adds **+10 to EACH ROW's revenue**
For example, let's say I have 100 rows so, +10 will be added to every row
So $100 * 10 = 1000$

This part will overwrite my **entire Silver table** with updated values
Keeps partitioning intact

```
1 gold_df = df_test.groupBy("category").agg(
2     sum("revenue").alias("total_revenue"),
3     sum("quantity").alias("total_quantity")
4 )
5
6 gold_df.write.mode("overwrite").saveAsTable("gold_sales")
```

✓ 8 sec - Command executed in 8 sec 77 ms by Ahmed Fabric on 5/2/2026, 5:27:20 PM

Next, I rebuilt the gold again

>  Spark jobs (12 of 12 succeeded)  Resources

Next:

- I'll refresh semantic model
- And refresh dashboard

If numbers change → LIVE connection confirmed

So, it's been verified that the pipeline is connected and the dashboard is showing us live values!

Before



After

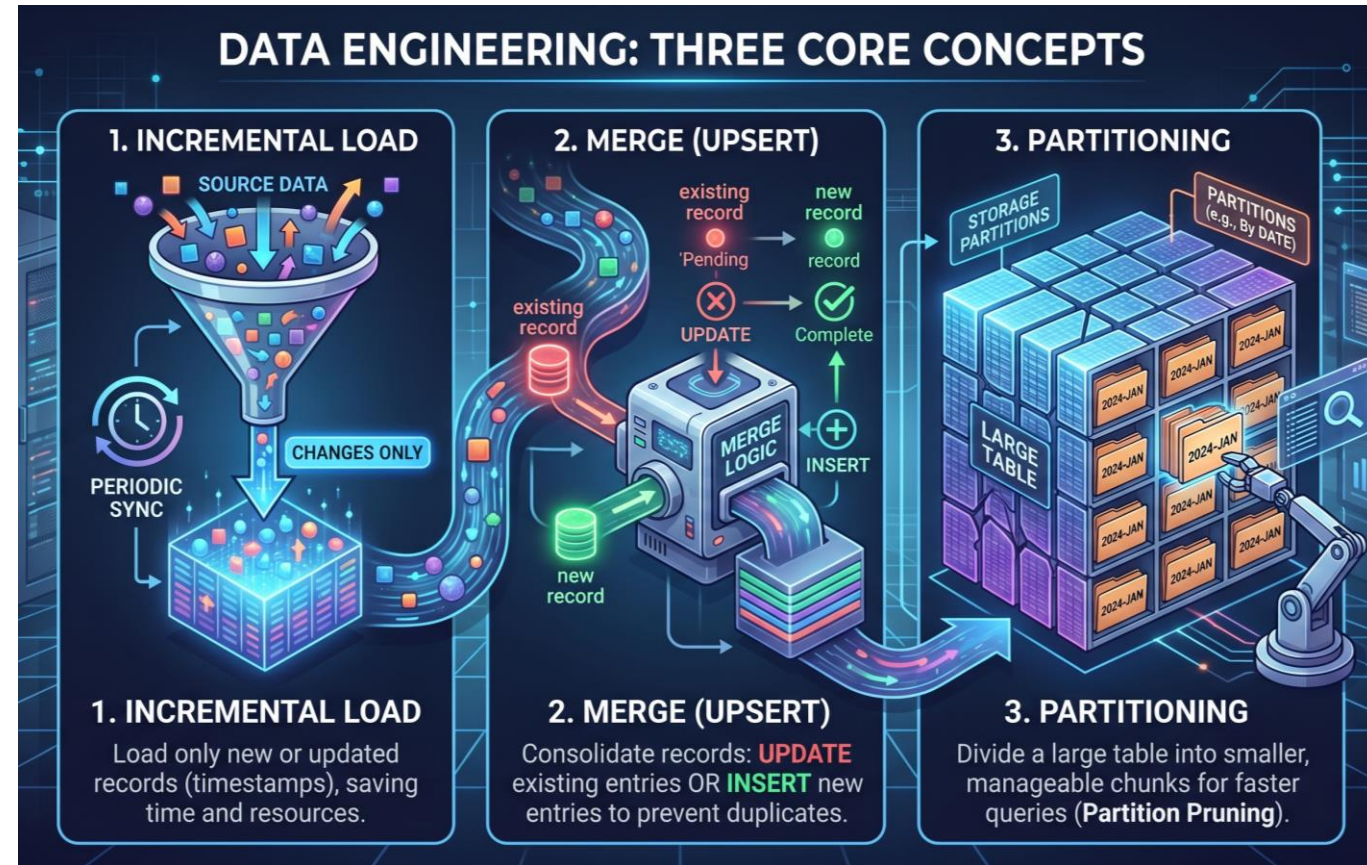


Congratulations, just built a complete end to end Fabric Pipeline with,

- Incremental Load
- Merge
- Partitioning

Step by step overview,

- Historical Data
- Incremental Load
- Merge
- Partitioning
- Business Aggregation
- Power BI Dashboard



Architecture

